SENIOR ARCHITECT MASTERCLASS

RAG

7 Topics across 7 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

Core RAG Pipeline

7 steps: document to answer

> Imagine you have a university exam tomorrow, but the exam is open-book. You do not need to memorize everything -- you just need to FIND the right page quickly and read the answer. RAG works the same way. Instead of hoping the LLM memorized the answer during training, you FIND the relevant documents and hand them to the LLM right before it answers.

1. End-to-End Pipeline

📖 DEEP TEACH

```
THE 7 STEPS (think of a librarian helping you):

Step 1 -- INGEST (librarian receives new books):
Load your documents into the system. PDFs, web pages, databases, emails -- any source.

Step 2 -- CHUNK (librarian cuts books into flash cards):
Your documents are too long to fit in the LLM's 'brain' at once. So you split them into small pieces -- like cutting a textbook
into individual flash cards. Each card is 500-1000 words.

Step 3 -- EMBED (librarian organizes cards by MEANING, not alphabetically):
Each flash card gets converted into a list of numbers (called a 'vector'). Cards with similar meanings get similar numbers.
'Machine learning' and 'AI algorithms' get numbers that are close together. 'Chocolate cake recipe' gets numbers that are far away.

Step 4 -- INDEX (librarian files the cards in a special cabinet):
Store all these numbered cards in a vector database. Think of it as a filing cabinet organized by MEANING instead of by alphabet.

Step 5 -- RETRIEVE (librarian searches for the right cards):
User asks a question. The question also gets converted into numbers. Then you search the filing cabinet for cards with the closest
numbers. Like asking the librarian: 'find me everything related to refund policies.'

Step 6 -- AUGMENT (librarian hands you the relevant pages):
Take the found cards and stuff them into the LLM's prompt: 'Here are the relevant documents. Using ONLY these, answer the
question.'

Step 7 -- GENERATE (you read the pages and write your answer):
The LLM reads the provided context and writes an answer. The answer is grounded in YOUR documents, not hallucinated from training
data.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Imagine you are taking an open-book exam. Your grade depends on TWO things: (1) Did you find the right pages? (2) Did you read them
correctly? RAG is the same.

1. START SIMPLE: ChromaDB (free vector database you install with pip) + OpenAI embeddings + recursive chunking. You can have a
working RAG system in 1 hour.
2. 80/20 RULE: 80% of your answer quality comes from chunking and retrieval. Fix how you cut documents and how you search FIRST. Do
not buy fancy tools before fixing the basics.
3. SPEED: The whole pipeline takes about 3 seconds. Embedding 50ms + searching 30ms + reranking 200ms + LLM answer 2 seconds.
4. COST: About $0.01-0.05 per question. At 100K questions per day, that is $1K-5K per day.

WAR STORY: A team built a fancy RAG system with GraphRAG and reranking. Their answer quality was terrible (60% accurate). Root
cause? Their chunks were cutting sentences in half. 'The refund policy is' was in one chunk and '30 days' was in the next. The
retriever found 'refund policy' but missed '30 days.' Fixed the chunking, accuracy jumped to 85%. Then they added the fancy stuff
and hit 93%. Always fix the basics first.
```

📋 POCKET NOTES

```
7 Steps: Ingest -> Chunk -> Embed -> Index -> Retrieve -> Augment -> Generate
Analogy: open-book exam (find the page, read the answer)
Chunk: 500-1000 words per piece, with 50-100 word overlap
Speed: ~3 seconds end-to-end
```

```
Cost: $0.01-0.05 per question
Quality lever #1: how you chunk (fix this FIRST)
Quality lever #2: how you search (hybrid > just embeddings)
Quality lever #3: reranking (double-check the top results)
Start with: ChromaDB + OpenAI embeddings + recursive chunking
```

🎯 INTERVIEW PREP

SAY THIS: 'I optimize RAG in priority order: chunking first, then retrieval strategy, then reranking. I have seen teams invest in GraphRAG while their basic chunking was splitting sentences in half. Fix fundamentals before adding complexity.'

Q: RAG vs fine-tuning -- when each?
A: 'RAG when knowledge changes frequently or you need citations. Fine-tuning when you need to change the model's behavior (tone, format). Best approach: fine-tune for format + RAG for knowledge.'

Q: Your RAG hallucinates 15% of the time. Debug process?
A: 'Isolate the step: (1) Wrong docs retrieved? Fix chunking/embedding. (2) Right docs but wrong answer? Chunks too big (noise) or too small (missing context). (3) Context is correct but LLM ignores it? Strengthen prompt with ONLY use context + citation requirement.'

⌨️ CODE

Python

```python
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

docs = ["Refund: 30 days. Electronics: 15 days. Must be unused.",
        "Warranty: 1 year for defects. Damage NOT covered.",
        "Pricing: Pro $49/mo. Enterprise $199/mo. Annual: 20% off."]
chunks = RecursiveCharacterTextSplitter(chunk_size=200,chunk_overlap=30).create_documents(docs)
vs = Chroma.from_documents(chunks, OpenAIEmbeddings(model="text-embedding-3-small"))
retrieved = vs.as_retriever(search_kwargs={"k":3}).invoke("Return electronics after 20 days?")
context = "\
".join([d.page_content for d in retrieved])

answer = (ChatPromptTemplate.from_template(
    "Using ONLY this context, answer. If not found, say 'I do not know.'\
Context:{c}\
Q:{q}")
    | ChatOpenAI(model="gpt-4") | StrOutputParser()
).invoke({"c":context,"q":"Can I return electronics after 20 days?"})
print(answer)
```

📺 OUTPUT

```
No, electronics have a 15-day return window, shorter than the 30-day standard.

# The LLM read YOUR documents and found the right answer.
# Without RAG, it would have guessed (maybe wrong).
```

Chunking Strategies

How you cut = how good your answers

> Imagine you have a textbook and you need to make flash cards. If you cut a page in the middle of a sentence -- 'The refund policy is' on one card and '30 days for unused items' on another -- neither card is useful alone. Good chunking = smart cutting that keeps complete ideas together.

2. Fixed + Recursive + Semantic + Parent-Child

📖 DEEP TEACH

4 WAYS TO CUT YOUR DOCUMENTS:

1. FIXED-SIZE (the dumb way):
Imagine cutting a rope into equal 100cm pieces. Fast and easy. But what if the knot is at position 99? You cut through it.

Fixed chunking splits every N characters regardless of content. It is fast but often cuts mid-sentence. Use it only for very uniform text like log files.

2. RECURSIVE (the smart default -- use this 90% of the time):
Imagine a smart editor reading your document. First, they try to split at paragraph breaks. If a paragraph is still too big, they split at sentence boundaries. If a sentence is somehow still too long, they split at word boundaries.

It respects the document's natural structure. Paragraphs stay together. Sentences stay complete. This is what you should use for almost everything.

3. SEMANTIC (the expensive genius):
Imagine a human reader who says 'the topic changed here.' They UNDERSTAND the content and split where the MEANING shifts.

How it works: embed every sentence, then compare neighboring sentences. Where the similarity DROPS = the topic changed = split here.

Highest quality but expensive (you embed every sentence during chunking). Use for high-value documents like legal contracts or medical papers.

4. PARENT-CHILD (the production standard):
This is the one most real systems use. The problem with all the above: small chunks give precise search results, but the LLM needs more context to write a good answer.

Solution: make TWO versions of your chunks.
  CHILDREN (small, 128 words): used for SEARCHING. Like an index card that says 'refund policy: 15 days for electronics.'
  PARENTS (big, 512 words): sent to the LLM. Like the full page that the index card points to.

You search the index cards (precise), then hand the LLM the full page (context-rich). Best of both worlds.

Imagine a library card catalog. The small card tells you exactly where to look. The big book gives you the full story.

🏗 ARCHITECT TIPS + WAR STORIES

1. RECURSIVE is your default. Do not overthink it. Use the others only when recursive is not good enough.
2. Chunk size sweet spot: 512 words for searching, 1024 words for sending to the LLM.
3. OVERLAP: Make each chunk share 10-20% of text with the next chunk. This prevents losing context at boundaries. Like flash cards where each card repeats the last sentence of the previous card.
4. Custom separators for code: split at class definitions and function definitions, not randomly.
5. LATE CHUNKING (2024, new): Embed the FULL document first, THEN split. Each chunk 'remembers' the whole document. Solves the 'It' problem: if a chunk says 'It was founded in 1998,' the chunk knows 'It' refers to 'Acme Corp' from page 1. Requires special embedding models (Jina 8K+).
6. COST: Recursive costs $0.001 per document. Semantic costs $0.10 per document (100x more). At 1 million documents: $1K vs $100K.

📋 POCKET NOTES

Fixed: cut every N characters. Fast, dumb, splits sentences.
Recursive: paragraphs -> sentences -> words. THE DEFAULT (90%).

Semantic: embed sentences, split at topic changes. Expensive but best quality.
Parent-Child: small chunks for search, big chunks for LLM. PRODUCTION STANDARD.
Analogy: library index cards (small, precise) pointing to full book pages (big, context-rich)
Chunk size: 512 for search, 1024 for LLM. Overlap: 10-20%.
Late Chunking: embed full doc first, then split. Solves pronoun problem. New (2024).
Cost: recursive $0.001/doc vs semantic $0.10/doc

🎯 INTERVIEW PREP

SAY THIS: 'I default to recursive chunking for 90% of cases. In production, I use parent-child: small chunks for precise search, big parent chunks for the LLM context. This solves the fundamental tension between search precision and generation quality.'

Q: How do you determine optimal chunk size?
A: 'Empirically. I test 256, 512, and 1024 on my eval set, measuring retrieval precision and answer quality. Smaller = more precise search but less context. I usually land on 512 for search with parent chunks of 1024.'

Q: Why not just use big chunks for everything?
A: 'A 1000-word chunk about 3 topics produces an embedding that is the AVERAGE of all 3 topics. It matches none of them well. A 128-word chunk about 1 topic produces a laser-focused embedding. But 128 words is not enough context for the LLM. Parent-child gives both.'

⌨️ CODE

Python

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.storage import InMemoryStore
from langchain.retrievers import ParentDocumentRetriever

doc = ["""Acme refund: standard 30 days. Electronics 15 days.
Digital products non-refundable once activated.
Shipping covered for defects only."""]

# PARENT-CHILD: small for search, big for LLM
retriever = ParentDocumentRetriever(
    vectorstore=Chroma(collection_name="c",
        embedding_function=OpenAIEmbeddings(model="text-embedding-3-small")),
    docstore=InMemoryStore(),
    child_splitter=RecursiveCharacterTextSplitter(chunk_size=80),
    parent_splitter=RecursiveCharacterTextSplitter(chunk_size=300))
retriever.add_documents(doc)
results = retriever.invoke("Return electronics after 20 days?")
print(f"Searched small CHILDREN, returned big PARENT ({len(results[0].page_content)} chars)")
print(results[0].page_content[:100]+"...")
```

📺 OUTPUT

```
Searched small CHILDREN, returned big PARENT (198 chars)
Acme refund: standard 30 days. Electronics 15 days. Digital products non-refundable...

# Child matched on 'Electronics 15 days' (precise)
# But the LLM gets the FULL policy with ALL context
```

Embeddings + Vector DBs

Turn words into numbers, search by meaning

> Imagine you could organize a library not alphabetically, but by MEANING. All books about 'cooking' would be near each other, even if one is titled 'Italian Cuisine' and another is 'Quick Weeknight Meals.' Embeddings do this: they turn text into numbers where similar meanings have similar numbers. Vector databases are the shelves that hold these numbered books.

3. 6 Models + 7 Databases

📖 DEEP TEACH

```
EMBEDDINGS -- HOW THEY WORK:
Take any text: 'Machine learning is cool'
Convert it to a list of numbers: [0.12, -0.34, 0.56, ...] (1536 numbers)

Now take: 'AI algorithms are awesome'
Convert it: [0.11, -0.33, 0.55, ...] (very SIMILAR numbers!)

And: 'Chocolate cake recipe'
Convert it: [0.89, 0.42, -0.71, ...] (very DIFFERENT numbers)

When you search, you compare numbers. Close numbers = similar meaning. That is how the LLM 'understands' your question matches your
documents even when the exact words differ.

6 EMBEDDING MODELS (which one to pick):
1. OpenAI text-embedding-3-small: Best balance of quality and cost. $0.02 per million words. Start here.
2. OpenAI text-embedding-3-large: Highest quality API model. 6x more expensive. For when quality is everything.
3. all-MiniLM-L6-v2: FREE. Runs on your computer. Good enough for prototypes.
4. BGE-large: FREE. Runs on your computer. Almost as good as OpenAI.
5. Jina v3: Supports long documents (8K words). Good for late chunking.
6. Cohere Embed v3: Great for multiple languages.

SHRINK TRICK (Matryoshka):
OpenAI models let you use fewer numbers per embedding: 1536 -> 256.
This saves 6x storage space with only 5% quality loss. Like compressing a photo.

7 VECTOR DATABASES (where to store embeddings):
Prototype: ChromaDB (install with pip, works in memory, zero setup)
Already use Postgres: pgvector extension (no new database to manage)
Production: Qdrant (fast, great filters, self-hosted)
Huge scale (billions): Milvus or Pinecone (managed, no ops)

WARNING -- LOCK-IN:
If you change your embedding model later, you must re-embed ALL your documents. Choose carefully upfront.
```

🏗 ARCHITECT TIPS + WAR STORIES

1. Start with OpenAI text-embedding-3-small. Best value.
2. Use the Matryoshka trick: 256 dimensions instead of 1536 = 6x less storage, 95% quality.
3. ALWAYS test on YOUR data. The leaderboard rankings are for general text. Your medical documents or legal contracts might behave differently.
4. pgvector works great under 10 million documents. After that, it slows down. Qdrant handles 50 million easily.
5. Storage math: 10 million documents x 1536 numbers x 4 bytes = 60 GB. With 256 numbers: only 10 GB.
6. Changing embedding models is the most expensive operation in a RAG system. Like switching from metric to imperial in a factory -- everything needs to be remeasured.

📋 POCKET NOTES

Embeddings: turn text into numbers. Similar meaning = similar numbers.
Models: OpenAI-small (best value) | MiniLM (free) | BGE (free+quality)
Matryoshka trick: 1536 -> 256 dimensions = 6x savings, 95% quality
Databases: ChromaDB (prototype) | pgvector (have Postgres) | Qdrant (production) | Pinecone (managed)

```
Storage: 10M docs x 1536d = 60GB | 10M x 256d = 10GB
Lock-in: changing embedding model = re-embed EVERYTHING
pgvector: great under 10M, slows after
Always benchmark on YOUR data, not leaderboard
Embedding cost: 10M docs = ~$100 one-time
```

🎯 INTERVIEW PREP

SAY THIS: 'I use OpenAI text-embedding-3-small with Matryoshka dimension reduction to 256. This gives the best cost-quality ratio -- 6x storage savings with only 5% quality loss. For databases, I match to scale: pgvector under 10M vectors, Qdrant for production, Pinecone for zero-ops.'

Q: How do you handle embedding model migration?
A: 'It is the most expensive RAG operation. I dual-write new embeddings while keeping old ones running. Shadow mode: query both, compare results. Atomic cutover when confidence is high. Budget: re-embedding 10M docs costs about $100 plus pipeline downtime.'

⌨️ CODE

Python

```python
from openai import OpenAI
import numpy as np
client = OpenAI()

texts = ["Machine learning uses data to find patterns",
         "AI algorithms learn from datasets",
         "Weather forecast predicts rain"]
resp = client.embeddings.create(model="text-embedding-3-small",input=texts)
embs = [r.embedding for r in resp.data]

def cos(a,b): return np.dot(a,b)/(np.linalg.norm(a)*np.linalg.norm(b))
print(f"ML vs AI:      {cos(embs[0],embs[1]):.3f} (HIGH = similar meaning!)")
print(f"ML vs Weather: {cos(embs[0],embs[2]):.3f} (LOW = different topics)")
```

📺 OUTPUT

```
ML vs AI:      0.912 (HIGH = the model knows these are related!)
ML vs Weather: 0.234 (LOW = the model knows these are unrelated)

# Even though 'machine learning' and 'AI algorithms' use different words,
# the embeddings are close because the MEANING is similar.
```

Retrieval Strategies

Dense + Sparse + Hybrid + Reranking

> Imagine you are searching a library for books about 'automobile safety.' A smart librarian (dense search) finds books about 'car crashes' and 'vehicle protection' -- they understand the MEANING. But they might miss a book literally titled 'Automobile Safety Standards' because they were thinking about meaning, not exact words. A keyword search (sparse) would find that exact title but miss 'car crashes.' Using BOTH = best results.

4. Hybrid Search + Reranking Pipeline

📖 DEEP TEACH

```
4 SEARCH METHODS (use them together):

1. DENSE SEARCH (meaning-based):
Convert question to numbers (embedding). Find documents with the closest numbers.
'automobile' matches 'car' because they MEAN the same thing.
BUT: 'error code 404' might NOT match 'HTTP 404' because the embeddings focus on meaning, and error codes are not really about
'meaning.'

2. SPARSE SEARCH / BM25 (keyword-based):
Old-school text matching. Count how many words overlap.
'error code 404' PERFECTLY matches '404 error code found.'
BUT: 'automobile' does NOT match 'car' because they are different words.

BM25 is free (no embeddings needed) and surprisingly good for technical content.

3. HYBRID (both together -- THE PRODUCTION STANDARD):
Run BOTH searches. Merge results using a formula called RRF (Reciprocal Rank Fusion).
Think of it as asking two librarians -- one who understands meaning and one who matches exact words -- and combining their
recommendations.

Hybrid catches what each search alone would miss.

4. RERANKING (double-check the results):
After hybrid search finds 20 candidate documents, a RERANKER re-scores them.

Imagine: the two librarians gave you 20 books. Now a senior librarian reads the question AND each book carefully, and picks the
best 5.

The reranker is more accurate than the initial search because it sees the question AND the document TOGETHER (not just comparing
numbers). Adds about 200ms but significantly improves quality.

THE PRODUCTION PIPELINE:
Question -> hybrid search (find top 20) -> reranker (pick best 5) -> send to LLM
```

🏗 ARCHITECT TIPS + WAR STORIES

1. ALWAYS use hybrid in production. Pure dense search misses keyword matches. Pure BM25 misses synonyms. Together they cover each other's weaknesses.
2. BM25 is FREE and does not need embeddings. Never skip it.
3. Reranking adds 200ms but improves precision by 10-20%. Worth it for most applications.
4. For technical documentation and code, weight BM25 higher. Function names and error codes need exact matching.
5. For conversational queries, weight dense search higher. Users speak naturally, not in keywords.
6. ALWAYS measure: if your retrieval precision at top 5 results is below 80%, fix retrieval before touching the LLM prompt.

📋 POCKET NOTES

```
Dense: embedding similarity. 'automobile' matches 'car'. Misses exact terms.
Sparse/BM25: keyword matching. 'error 404' matches '404 error'. FREE.
Hybrid: dense + sparse + RRF merge. PRODUCTION STANDARD.
Reranking: senior librarian re-scores top 20 -> picks best 5. +10-20% precision. +200ms.
Pipeline: hybrid top-20 -> rerank top-5 -> LLM
```

Weight: code/technical = more BM25. Conversational = more dense.
Measure: precision@5 > 80% before optimizing LLM prompt

🎯 INTERVIEW PREP

SAY THIS: 'My standard pipeline is hybrid search with reranking. Dense for semantic understanding, BM25 for keyword precision, merged via RRF. The reranker re-scores the top 20 to surface the best 5 for the LLM.'

Q: Reranking adds 200ms. Worth it?
A: 'For enterprise search where answer quality matters? Absolutely. For real-time chatbot where speed is everything? Maybe skip and invest in better chunking. I always benchmark with and without reranking on my eval set.'

⌨️ CODE

Python

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from rank_bm25 import BM25Okapi
import numpy as np

docs = ["Python error code 404 means page not found",
        "ML algorithms learn from training data",
        "HTTP 404 status indicates missing resource",
        "AI is transforming technology",
        "Debug Python errors by checking stack trace"]

# Dense search (by meaning)
vs = Chroma.from_texts(docs, OpenAIEmbeddings(model="text-embedding-3-small"))
dense = [d.page_content[:50] for d,_ in vs.similarity_search_with_score("Python 404 error",k=3)]
print("DENSE (meaning):",dense)

# BM25 search (by keywords)
bm25 = BM25Okapi([d.lower().split() for d in docs])
scores = bm25.get_scores("python 404 error".split())
sparse = [docs[i][:50] for i in np.argsort(scores)[::-1][:3]]
print("BM25 (keywords):",sparse)

print("\
Dense MISSED exact '404'. BM25 nailed it.")
print("Hybrid = best of both.")
```

📺 OUTPUT

```
DENSE (meaning): ['ML algorithms learn...', 'AI is transforming...', 'Python error code 404...']
BM25 (keywords):  ['Python error code 404...', 'Debug Python errors...', 'HTTP 404 status...']

Dense MISSED exact '404' (ranked it 3rd!). BM25 nailed it (ranked 1st).
Hybrid = best of both.
```

Advanced RAG

Multi-query, CRAG, Agentic, GraphRAG

> Basic RAG is like asking one librarian one question. Advanced RAG is like having a TEAM of librarians: one rephrases your question 3 different ways to find more books, another checks if the books are actually relevant before handing them to you, and a third can walk between shelves to connect information across multiple books.

5. Multi-Query + CRAG + Agentic + GraphRAG

📖 DEEP TEACH

6 ADVANCED PATTERNS (each fixes a specific problem):

1. MULTI-QUERY (fix: 'I cannot find enough documents'):
Imagine you search Google for 'how to reset my password.' You find 2 results. Then you search 'forgot password recovery' and find 3 MORE results. Then 'change account credentials' finds 2 more. By asking the same question 3 different ways, you find way more relevant documents.

Multi-query: LLM rephrases your question 3 ways, searches for each, combines all results. +15-25% more documents found.

2. HyDE (fix: 'my short question does not match long documents'):
Imagine searching for a research paper. Your question is 10 words. The papers are 5000 words. Short questions and long documents have different embeddings.

HyDE trick: ask the LLM to write a FAKE answer first. The fake answer is long, like a real document. Search using the fake answer's embedding instead of the short question's. The fake answer is closer in 'number space' to real documents. +10-20% on complex questions.

3. SELF-RAG (fix: 'I waste money searching for simple questions'):
Imagine someone asks your librarian 'what time is it?' The librarian does NOT need to search any books. They just answer.

Self-RAG: the LLM first decides IF it even needs to search documents. Simple questions (hi, what is 2+2) skip retrieval entirely. Saves 30-50% of retrieval costs.

4. CRAG / Corrective RAG (fix: 'the retrieved documents are irrelevant'):
Imagine your librarian finds 5 books, but 2 are about a completely different topic. If they hand all 5 to you, those 2 irrelevant books confuse you.

CRAG: the LLM GRADES each retrieved document -- 'is this actually relevant? YES or NO.' Irrelevant ones are thrown away BEFORE the LLM tries to answer. Reduces hallucination by 20-40%.

5. AGENTIC RAG (fix: 'I need different search strategies for different questions'):
Imagine one question needs your internal database, another needs a Google search, and a third needs both. A fixed pipeline cannot adapt.

Agentic RAG: an AI agent DECIDES how to search for each question. It can search multiple databases, do web searches, query SQL databases, and combine results. It adapts its strategy per question.

6. GRAPHRAG (fix: 'I need to connect information across documents'):
Imagine someone asks 'Who does Bob ultimately report to?' The answer requires following a chain: Bob reports to Alice. Alice reports to Carol (CEO). This chain spans 3 different documents. Regular search finds individual documents but cannot follow chains.

GraphRAG: builds a knowledge graph (people, projects, relationships) from your documents. Then traverses the graph to follow connections. 'Bob -> reports_to -> Alice -> reports_to -> Carol (CEO)'

🏗 ARCHITECT TIPS + WAR STORIES

1. MULTI-QUERY: easiest to implement, biggest immediate impact. Start here.
2. CRAG: catches irrelevant documents before they confuse the LLM. Great for quality.
3. SELF-RAG: best for saving money. Simple questions skip retrieval entirely.
4. GRAPHRAG: when more than 20% of your questions need connections across documents.
5. Implement them as LangGraph state machines with conditional edges.
6. Do NOT add all patterns at once. Each adds latency and cost. Add the one that fixes your measured weakness.

```
DECISION: What is your problem?
  Not finding enough docs? -> multi-query
  Finding irrelevant docs? -> CRAG
  Too expensive? -> Self-RAG
  Need connections across docs? -> GraphRAG
  Need different search per question? -> Agentic RAG
```

📋 POCKET NOTES

Multi-Query: rephrase question 3 ways, search each, combine. +15-25% recall.
HyDE: generate fake answer, search with that. +10-20% complex Qs.
Self-RAG: skip retrieval for simple Qs. Save 30-50% cost.
CRAG: grade retrieved docs, throw away irrelevant. -20-40% hallucination.
Agentic: AI agent decides HOW to search per question.
GraphRAG: knowledge graph from docs. Multi-hop connections.
Priority: multi-query (easy win) > CRAG (quality) > Self-RAG (cost) > GraphRAG (connections)
Do NOT add all at once. Fix your measured weakness.

🎯 INTERVIEW PREP

SAY THIS: 'I layer advanced patterns based on the specific failure I am seeing. Low recall? Multi-query. Hallucination? CRAG. Cost too high? Self-RAG gating. Multi-hop questions? GraphRAG. I diagnose first, then add the right fix.'

Q: GraphRAG -- when is it worth the effort?
A: 'When more than 20% of questions require information from multiple documents connected by relationships. Entity extraction costs about $2K for 100K documents. The precision improvement on relationship queries is typically 20-40%. If your users mostly ask simple factual questions, GraphRAG is overkill.'

⌨ CODE

Python

```python
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model="gpt-4")
docs = ["Reset password: Settings > Security > Change.",
        "Recovery: SMS or backup codes.",
        "2FA: Settings > Security > Two-Factor.",
        "Pricing: $29/month Basic."]
vs = Chroma.from_texts(docs, OpenAIEmbeddings(model="text-embedding-3-small"))

q = "Forgot credentials, how to get back in?"

# MULTI-QUERY: rephrase 3 ways, search each
variations = (ChatPromptTemplate.from_template(
    "Generate 3 search queries for: {q}\
One per line.") | llm | StrOutputParser()
).invoke({"q":q}).strip().split("\
")
all_docs = set()
for query in [q]+variations[:3]:
    for r in vs.similarity_search(query,k=2): all_docs.add(r.page_content[:40])
print(f"Multi-query found {len(all_docs)} docs (vs 2 with single search)")

# CRAG: grade each doc -- relevant or not?
for doc in list(all_docs):
    grade = (ChatPromptTemplate.from_template(
        "Is this relevant to '{q}'?\
Doc: {d}\
YES or NO only.") | llm | StrOutputParser()
    ).invoke({"q":q,"d":doc})
    print(f"  [{'KEEP' if 'YES' in grade.upper() else 'TOSS'}] {doc}")
```

📺 OUTPUT

```
Multi-query found 3 docs (vs 2 with single search)
  [KEEP] Reset password: Settings > Security...
  [KEEP] Recovery: SMS or backup codes...
  [TOSS] Pricing: $29/month Basic...

# Multi-query: found more relevant docs by rephrasing
# CRAG: threw away the irrelevant pricing doc before it could confuse the LLM
```

Multimodal RAG

Images, tables, PDFs

> Real documents are not just text. A financial report has charts. A medical record has X-ray images. A product manual has diagrams. Multimodal RAG handles ALL of these -- not just the words on the page.

6. Tables + Images + Vision-Augmented

📖 DEEP TEACH

```
THE CHALLENGE:
Imagine your RAG system reads a financial report. The text says 'see Figure 3 for revenue breakdown.' But your system cannot see
Figure 3. It is a chart. Your system only reads text. So it skips the most important information.

3 WAYS TO FIX THIS:

1. CONVERT EVERYTHING TO TEXT (simplest, works today):
Tables: convert to markdown format. The LLM understands markdown tables very well.
  | Product | Revenue |
  |---|---|
  | Widget A | $2.1M |

Images/Charts: ask GPT-4o to DESCRIBE the image. 'This is a bar chart showing Widget A at $2.1M and Widget B at $1.8M.' Store that
description as a text chunk, just like any other text.

This is like having someone describe a painting to a blind person. Not perfect, but 90% of the value.

2. MULTIMODAL EMBEDDINGS (embed images directly):
Use models (CLIP, Jina-CLIP) that can turn BOTH text and images into the same type of numbers. An image of a cat and the text 'a
cat' get similar numbers. Then you can search images the same way you search text.

3. VISION-AUGMENTED (send images to the LLM):
Send the actual chart image to GPT-4o or Claude. The LLM 'looks at' the chart and reads the data directly. Most accurate but most
expensive.

PRODUCTION APPROACH:
Convert tables to markdown + describe images with GPT-4o + embed everything as text.
90% of the value with your existing text-only infrastructure.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
1. Tables -> markdown. LLMs understand markdown tables well. Just format them with | and ---.
2. Image description costs about $0.01 per image. For 10,000 images = $100. One-time cost.
3. Scanned PDFs (where text is actually a picture): use OCR (Tesseract or AWS Textract) first.
4. The Unstructured library handles PDF/DOCX/HTML parsing automatically. pip install unstructured. It is the Swiss Army knife for
document processing.
5. For critical charts, BOTH describe the chart AND send the original image to the LLM. Belt and suspenders.
```

📋 POCKET NOTES

```
3 Approaches: text extraction | multimodal embeddings | vision-augmented
Tables -> markdown format (LLMs understand this well)
Images -> GPT-4o describes them (~$0.01/image)
Scanned PDFs -> OCR (Tesseract, AWS Textract)
Multimodal embeddings: CLIP, Jina-CLIP. Same numbers for text+images.
Production: convert everything to text. 90% of the value.
Unstructured library: Swiss Army knife for document parsing.
pip install unstructured
```

🎯 INTERVIEW PREP

SAY THIS: 'For multimodal RAG, I use a pragmatic approach. Tables become markdown. Images get GPT-4o descriptions. Everything becomes text embeddings in the same vector space. This gives 90% of the value with existing text-only infrastructure.'

Q: User asks about a chart in a PDF. How does your system handle it?
A: 'Pipeline: Unstructured library extracts the chart image. GPT-4o generates a detailed description including all data points. That description is chunked and embedded like any other text. For maximum accuracy, I also send the original chart image alongside the text context to the LLM.'

⌨ CODE

Python

```python
# Convert tables to markdown for RAG
table = [["Product","Revenue"],["Widget A","$2.1M"],["Widget B","$1.8M"]]
md = "| " + " | ".join(table[0]) + " |\
|---|---|\
"
for row in table[1:]:
    md += "| " + " | ".join(row) + " |\
"
print("TABLE AS MARKDOWN (LLM reads this easily):")
print(md)

# Describe images with GPT-4o vision
from openai import OpenAI
client = OpenAI()
desc = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role":"user","content":[
        {"type":"text","text":"Describe this chart for a search system."},
        {"type":"text","text":"[Simulated: bar chart Widget A $2.1M, Widget B $1.8M]"}]}]
).choices[0].message.content
print(f"IMAGE -> TEXT: {desc[:100]}...")
print("\
Both become regular text chunks in the same vector store.")
```

📺 OUTPUT

```
TABLE AS MARKDOWN (LLM reads this easily):
| Product | Revenue |
|---|---|
| Widget A | $2.1M |
| Widget B | $1.8M |

IMAGE -> TEXT: Bar chart comparing Q3 revenue. Widget A leads at $2.1M...

Both become regular text chunks in the same vector store.
```

RAG Evaluation (RAGAS)

4 metrics: is your RAG actually working?

Imagine you built a study assistant for students. How do you know if it WORKS? You need to check 4 things: Did it find the right textbook pages? Did it find ALL the relevant pages? Did it answer using ONLY those pages (not making stuff up)? Did it actually answer the question? RAGAS checks all 4.

7. Precision + Recall + Faithfulness + Relevance

📖 DEEP TEACH

```
4 THINGS TO MEASURE:

1. CONTEXT PRECISION (> 80%): 'Did the librarian find the RIGHT books?'
Imagine the librarian brings you 3 books for your refund question. One is about refunds. One is about weather. One is about cooking. Only 1 out of 3 is relevant = 33% precision. That is terrible.
FIX: Better chunking. Better search. Add reranking.

2. CONTEXT RECALL (> 75%): 'Did the librarian find ALL the relevant books?'
There are 3 books about refunds in the library. The librarian only found 1 = 33% recall. They missed 2 important books.
FIX: Add hybrid search (BM25 catches what embeddings miss). Multi-query retrieval.

3. FAITHFULNESS (> 90% -- THE MOST CRITICAL): 'Did the student answer using ONLY what the books say?'
The book says '30-day refund.' The student writes '60-day refund.' They made it up. That is unfaithful = HALLUCINATION.
This is the #1 metric. If below 90%, your system is making things up and users will get wrong answers.
FIX: Stronger prompt ('ONLY use the provided context'). Citation requirement. CRAG to filter bad docs.

4. ANSWER RELEVANCE (> 85%): 'Did the student actually answer the question?'
Question: 'What is the refund policy?' Answer: 'We have great products!' That does not answer the question.
FIX: Better generation prompt. Better model.

DIAGNOSTIC MAP (memorize this):
  Low precision -> your SEARCH is sloppy. Fix chunking or add reranking.
  Low recall -> your SEARCH is missing docs. Add hybrid search or multi-query.
  Low faithfulness -> your LLM is MAKING STUFF UP. Fix the prompt, add citations.
  Low relevance -> your LLM is OFF TOPIC. Fix the generation prompt or use a better model.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
1. FAITHFULNESS is the #1 metric. If below 90%, stop everything and fix it. Your system is lying to users.
2. Build an eval set of 200+ real questions with known correct answers. Include questions that CANNOT be answered from your docs (system should say 'I do not know').
3. Run RAGAS in your CI/CD pipeline. If someone changes a prompt and faithfulness drops, the deploy is BLOCKED.
4. For ongoing monitoring: sample 1% of production traffic, score it automatically, track weekly trends.
5. Refresh your eval set every 3 months with new real user questions.

WAR STORY: A RAG system had 92% answer relevance (answers sounded good) but only 65% faithfulness. The answers SOUNDED right but contained made-up facts. The LLM was ignoring the context and using its training data. Fix: added 'ONLY use the provided context. If the answer is not in the context, say I do not know.' plus a citation requirement. Faithfulness jumped to 94%.
```

📋 POCKET NOTES

```
4 RAGAS Metrics:
  Precision > 80%: found the RIGHT docs? (fix: chunking/reranking)
  Recall > 75%: found ALL relevant docs? (fix: hybrid/multi-query)
  Faithfulness > 90%: answer from docs ONLY? (fix: prompt/citations) -- CRITICAL
  Relevance > 85%: answer addresses question? (fix: prompt/model)
Analogy: librarian finds books (precision+recall), student answers from books only (faithfulness), student answers the actual question (relevance)
Eval set: 200+ samples + unanswerable questions
CI/CD gate: faithfulness < 90% = block deploy
```

```
Online: sample 1% production, track weekly
pip install ragas deepeval
```

🎯 INTERVIEW PREP

SAY THIS: 'Faithfulness is my gating metric. Below 90% means the system is hallucinating -- making things up. I maintain 200+ eval samples, run as a CI/CD gate on every prompt change, and sample 1% of production traffic weekly.'

Q: Faithfulness is 92% but users still complain about wrong answers.
A: '8% error rate means 1 in 12 answers is wrong. Users notice. Three fixes: add CRAG to grade retrieved docs before generation, require citations so users can verify, and check if errors cluster on specific topics -- those topics need better documentation or specialized retrieval.'

Q: How often should you run RAGAS?
A: 'Three levels: CI/CD on every prompt change, weekly batch on full eval set, daily 1% production sampling with automated alerts if any metric drops more than 3%.'

⌨ CODE

Python

```python
from ragas import evaluate
from ragas.metrics import context_precision, context_recall, faithfulness, answer_relevancy
from datasets import Dataset

eval_data = {
    "question": ["Refund policy?", "Pro plan cost?", "Return electronics after 20 days?"],
    "answer": ["Full refund 30 days.", "$49/mo, 20% annual discount.", "No, 15-day window."],
    "contexts": [["Refund: 30 days unused."],["Pro $49/mo. Annual 20% off."],["Electronics: 15 days."]],
    "ground_truth": ["30 days unused.", "$49/mo, 20% annual.", "No, 15 days."]
}

results = evaluate(
    Dataset.from_dict(eval_data),
    metrics=[context_precision, context_recall, faithfulness, answer_relevancy])

for metric, score in results.items():
    if isinstance(score, float):
        s = 'PASS' if score > 0.8 else 'WARN' if score > 0.6 else 'FAIL'
        print(f"  [{s}] {metric}: {score:.2f}")
```

📺 OUTPUT

```
  [PASS] context_precision: 0.94
  [PASS] context_recall: 0.89
  [PASS] faithfulness: 0.97
  [PASS] answer_relevancy: 0.92

# All above 80% = healthy RAG system.
# Faithfulness 0.97 = minimal hallucination. The system answers from docs, not imagination.
```

LIBRARIES

langchain	RAG chains	`pip install langchain langchain-openai`
llama-index	RAG indexes	`pip install llama-index`
chromadb	Vector DB (prototype)	`pip install chromadb`
qdrant-client	Vector DB (production)	`pip install qdrant-client`
ragas	RAG evaluation	`pip install ragas`
unstructured	PDF/HTML parsing	`pip install unstructured`
rank_bm25	BM25 keyword search	`pip install rank_bm25`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```
%% ============================================================
%% D1: Full Production RAG System
%% ============================================================
%% Save as: diagrams/rag-production-system.mermaid

graph TB
    User([User Query]) --> Gateway[LiteLLM Gateway]

    Gateway --> Cache{Semantic Cache}
    Cache -->|HIT| Response([Response])
    Cache -->|MISS| Router[Complexity Router]

    Router -->|Simple| MiniGen[GPT-4o-mini Generator]
    Router -->|Complex| FullPipeline[Full RAG Pipeline]

    FullPipeline --> Hybrid[Hybrid Search]

    subgraph Retrieval
        Hybrid --> Dense[Dense Search - Embeddings]
        Hybrid --> Sparse[Sparse Search - BM25]
        Dense --> RRF[RRF Merge]
        Sparse --> RRF
    end

    RRF --> Reranker[Cohere Reranker - Top 5]
    Reranker --> Augment[Context Augmentation]
    Augment --> Generator[GPT-4o Generator]

    MiniGen --> Safety
    Generator --> Safety[Guardrails + PII Check]

    Safety --> Monitor[Monitor - Tokens, Latency, Cost]
    Monitor --> Cache
    Monitor --> Response

    subgraph Vector Store
        VDB[(Qdrant / pgvector)]
        Meta[Metadata Filters]
    end

    Dense --> VDB
    Meta --> VDB

    subgraph Embedding Pipeline
        Docs[Documents] --> Chunk[Recursive Chunker]
        Chunk --> Embed[Embedding Model]
        Embed --> Dedup[Dedup - Content Hash]
        Dedup --> VDB
    end

    style Gateway fill:#1f6feb,color:#fff
    style Cache fill:#d29922,color:#fff
    style Router fill:#8957e5,color:#fff
    style Safety fill:#da3633,color:#fff
    style Monitor fill:#3fb950,color:#fff
    style Response fill:#1f6feb,color:#fff
```

— POCKET NOTES —

1. End-to-End Pipeline

7 Steps: Ingest -> Chunk -> Embed -> Index -> Retrieve -> Augment -> Generate
Analogy: open-book exam (find the page, read the answer)
Chunk: 500-1000 words per piece, with 50-100 word overlap
Speed: ~3 seconds end-to-end
Cost: $0.01-0.05 per question
Quality lever #1: how you chunk (fix this FIRST)
Quality lever #2: how you search (hybrid > just embeddings)
Quality lever #3: reranking (double-check the top results)
Start with: ChromaDB + OpenAI embeddings + recursive chunking

2. Fixed + Recursive + Semantic + Parent-Child

Fixed: cut every N characters. Fast, dumb, splits sentences.
Recursive: paragraphs -> sentences -> words. THE DEFAULT (90%).
Semantic: embed sentences, split at topic changes. Expensive but best quality.
Parent-Child: small chunks for search, big chunks for LLM. PRODUCTION STANDARD.
Analogy: library index cards (small, precise) pointing to full book pages (big, context-rich)
Chunk size: 512 for search, 1024 for LLM. Overlap: 10-20%.

```
Late Chunking: embed full doc first, then split. Solves pronoun problem. New (2024).
Cost: recursive $0.001/doc vs semantic $0.10/doc
```

3. 6 Models + 7 Databases

```
Embeddings: turn text into numbers. Similar meaning = similar numbers.
Models: OpenAI-small (best value) | MiniLM (free) | BGE (free+quality)
Matryoshka trick: 1536 -> 256 dimensions = 6x savings, 95% quality
Databases: ChromaDB (prototype) | pgvector (have Postgres) | Qdrant (production) | Pinecone (managed)
Storage: 10M docs x 1536d = 60GB | 10M x 256d = 10GB
Lock-in: changing embedding model = re-embed EVERYTHING
pgvector: great under 10M, slows after
Always benchmark on YOUR data, not leaderboard
Embedding cost: 10M docs = ~$100 one-time
```

4. Hybrid Search + Reranking Pipeline

```
Dense: embedding similarity. 'automobile' matches 'car'. Misses exact terms.
Sparse/BM25: keyword matching. 'error 404' matches '404 error'. FREE.
Hybrid: dense + sparse + RRF merge. PRODUCTION STANDARD.
Reranking: senior librarian re-scores top 20 -> picks best 5. +10-20% precision. +200ms.
Pipeline: hybrid top-20 -> rerank top-5 -> LLM
Weight: code/technical = more BM25. Conversational = more dense.
Measure: precision@5 > 80% before optimizing LLM prompt
```

5. Multi-Query + CRAG + Agentic + GraphRAG

```
Multi-Query: rephrase question 3 ways, search each, combine. +15-25% recall.
HyDE: generate fake answer, search with that. +10-20% complex Qs.
Self-RAG: skip retrieval for simple Qs. Save 30-50% cost.
CRAG: grade retrieved docs, throw away irrelevant. -20-40% hallucination.
Agentic: AI agent decides HOW to search per question.
GraphRAG: knowledge graph from docs. Multi-hop connections.
Priority: multi-query (easy win) > CRAG (quality) > Self-RAG (cost) > GraphRAG (connections)
Do NOT add all at once. Fix your measured weakness.
```

6. Tables + Images + Vision-Augmented

```
3 Approaches: text extraction | multimodal embeddings | vision-augmented
Tables -> markdown format (LLMs understand this well)
Images -> GPT-4o describes them (~$0.01/image)
Scanned PDFs -> OCR (Tesseract, AWS Textract)
Multimodal embeddings: CLIP, Jina-CLIP. Same numbers for text+images.
Production: convert everything to text. 90% of the value.
Unstructured library: Swiss Army knife for document parsing.
pip install unstructured
```

7. Precision + Recall + Faithfulness + Relevance

```
4 RAGAS Metrics:
  Precision > 80%: found the RIGHT docs? (fix: chunking/reranking)
  Recall > 75%: found ALL relevant docs? (fix: hybrid/multi-query)
  Faithfulness > 90%: answer from docs ONLY? (fix: prompt/citations) -- CRITICAL
  Relevance > 85%: answer addresses question? (fix: prompt/model)
Analogy: librarian finds books (precision+recall), student answers from books only (faithfulness), student answers the actual question
(relevance)
Eval set: 200+ samples + unanswerable questions
CI/CD gate: faithfulness < 90% = block deploy
Online: sample 1% production, track weekly
pip install ragas deepeval
```

— INTERVIEW PREP —

1. End-to-End Pipeline

SAY THIS: 'I optimize RAG in priority order: chunking first, then retrieval strategy, then reranking. I have seen teams invest in GraphRAG while their basic chunking was splitting sentences in half. Fix fundamentals before adding complexity.'

Q: RAG vs fine-tuning -- when each?
A: 'RAG when knowledge changes frequently or you need citations. Fine-tuning when you need to change the model's behavior (tone, format). Best approach: fine-tune for format + RAG for knowledge.'

Q: Your RAG hallucinates 15% of the time. Debug process?
A: 'Isolate the step: (1) Wrong docs retrieved? Fix chunking/embedding. (2) Right docs but wrong answer? Chunks too big (noise) or too small (missing context). (3) Context is correct but LLM ignores it? Strengthen prompt with ONLY use context + citation requirement.'

2. Fixed + Recursive + Semantic + Parent-Child

SAY THIS: 'I default to recursive chunking for 90% of cases. In production, I use parent-child: small chunks for precise search, big parent chunks for the LLM context. This solves the fundamental tension between search precision and generation quality.'

Q: How do you determine optimal chunk size?
A: 'Empirically. I test 256, 512, and 1024 on my eval set, measuring retrieval precision and answer quality. Smaller = more precise search but less context. I usually land on 512 for search with parent chunks of 1024.'

Q: Why not just use big chunks for everything?
A: 'A 1000-word chunk about 3 topics produces an embedding that is the AVERAGE of all 3 topics. It matches none of them well. A 128-word chunk about 1 topic produces a laser-focused embedding. But 128 words is not enough context for the LLM. Parent-child gives both.'

3. 6 Models + 7 Databases

SAY THIS: 'I use OpenAI text-embedding-3-small with Matryoshka dimension reduction to 256. This gives the best cost-quality ratio -- 6x storage savings with only 5% quality loss. For databases, I match to scale: pgvector under 10M vectors, Qdrant for production, Pinecone for zero-ops.'

Q: How do you handle embedding model migration?
A: 'It is the most expensive RAG operation. I dual-write new embeddings while keeping old ones running. Shadow mode: query both, compare results. Atomic cutover when confidence is high. Budget: re-embedding 10M docs costs about $100 plus pipeline downtime.'

4. Hybrid Search + Reranking Pipeline

SAY THIS: 'My standard pipeline is hybrid search with reranking. Dense for semantic understanding, BM25 for keyword precision, merged via RRF. The reranker re-scores the top 20 to surface the best 5 for the LLM.'

Q: Reranking adds 200ms. Worth it?
A: 'For enterprise search where answer quality matters? Absolutely. For real-time chatbot where speed is everything? Maybe skip and invest in better chunking. I always benchmark with and without reranking on my eval set.'

5. Multi-Query + CRAG + Agentic + GraphRAG

SAY THIS: 'I layer advanced patterns based on the specific failure I am seeing. Low recall? Multi-query. Hallucination? CRAG. Cost too high? Self-RAG gating. Multi-hop questions? GraphRAG. I diagnose first, then add the right fix.'

Q: GraphRAG -- when is it worth the effort?
A: 'When more than 20% of questions require information from multiple documents connected by relationships. Entity extraction costs about $2K for 100K documents. The precision improvement on relationship queries is typically 20-40%. If your users mostly ask simple factual questions, GraphRAG is overkill.'

6. Tables + Images + Vision-Augmented

SAY THIS: 'For multimodal RAG, I use a pragmatic approach. Tables become markdown. Images get GPT-4o descriptions. Everything becomes text embeddings in the same vector space. This gives 90% of the value with existing text-only infrastructure.'

Q: User asks about a chart in a PDF. How does your system handle it?
A: 'Pipeline: Unstructured library extracts the chart image. GPT-4o generates a detailed description including all data points. That description is chunked and embedded like any other text. For maximum accuracy, I also send the original chart image alongside the text context to the LLM.'

7. Precision + Recall + Faithfulness + Relevance

SAY THIS: 'Faithfulness is my gating metric. Below 90% means the system is hallucinating -- making things up. I maintain 200+ eval samples, run as a CI/CD gate on every prompt change, and sample 1% of production traffic weekly.'

Q: Faithfulness is 92% but users still complain about wrong answers.
A: '8% error rate means 1 in 12 answers is wrong. Users notice. Three fixes: add CRAG to grade retrieved docs before generation, require citations so users can verify, and check if errors cluster on specific topics -- those topics need better documentation or specialized retrieval.'

Q: How often should you run RAGAS?
A: 'Three levels: CI/CD on every prompt change, weekly batch on full eval set, daily 1% production sampling with automated alerts if any metric drops more than 3%.'